

HopCharge HR Dashboard Codebase Guide

A plain-language walkthrough of every file in the **BAU Automation – Hopcharge** project — what it does and why it matters.

Folder documented

~/Desktop/BAU Automation- Hopcharge

Prepared

4 July 2026

What is this document?

This guide explains, in everyday language, what every file and folder in the HopCharge HR Dashboard project does. The HR Dashboard is an internal tool that automates recruitment and HR work end to end: it reads uploaded resumes, scores candidates (with rule-based logic and optionally AI), emails applicants, tracks their progress through hiring stages, manages the employee database after hiring, and even helps the team reach out to colleges for campus hiring.

Files are grouped by what they do rather than listed alphabetically, and each one is tagged by importance:

● **Core** — central to how the app works day to day ● **Supporting** — useful, secondary functionality

● **Minor** — small, technical, or rarely-touched

Contents

1. Application Core & Startup	App engine, database link, security
2. Resume Reading & Processing	Turning PDFs into candidate data
3. Candidate Pipeline & Records	Accept / reject / notes / employees
4. Candidate Communication	Emails, interview booking, status page
5. Admin, Settings & Reporting	Configurable rules and dashboards
6. Data Sync, Backup & Migration	Keeping the database and files in agreement
7. AI Candidate Scoring Module (<code>scoring/</code>)	12 files
8. College Outreach Module (<code>colleges/</code>)	13 files
9. Setup, Installers & Branding	How the app gets installed and looks
10. Data, Credentials & Log Folders	Where information is actually stored

Note on security: Several files (e.g. `neon.env` , `Gdrive service acc.json` , `Gforms service acc.json` , contents of `credentials/`) hold private passwords or login keys. This guide describes only their *purpose* — never their contents — and they should never be shared, emailed, or uploaded anywhere.

1 Application Core & Startup

These files are the "engine room" — they start the app, connect it to the database, define how data is structured, and keep sensitive information secure. Almost everything else in the project depends on these.

`app.py`

CORE

The engine room of the HR Dashboard web application. It starts up the website, wires together every feature (resume upload/parsing, sending recruitment and onboarding emails, tracking whether candidates opened their application form, Google Forms syncing, college outreach, candidate scoring, analytics), and defines every action the dashboard's buttons and pages can trigger — uploading resumes, downloading results, emailing a batch of candidates, and more. Any user interacting with the dashboard is ultimately triggering code in this file, making it the single most important file for daily operation.

`main.py`

SUPPORTING

A simple command-line way to run the resume parser without opening the web dashboard — point it at a folder of PDF resumes and it processes them all, saving results as JSON, CSV, and Excel files. Mainly useful for technical staff doing a bulk batch-parse from a terminal, since the web dashboard already offers the same parsing.

`config.py`

SUPPORTING

Holds fixed rules the resume parser uses to read PDFs correctly — how it recognizes section headings like "Education" or "Work Experience," and patterns for finding emails, phone numbers, and LinkedIn links. It also decides when a resume needs OCR (image-to-text conversion) because the PDF text can't be read directly.

`database.py`

CORE

Manages the connection to the company's cloud database (Neon, a hosted PostgreSQL service) where candidate, employee, and college data is permanently stored. It automatically reads the secret connection details, keeps the connection alive (the cloud database "sleeps" when unused and this code wakes it back up), and provides a safe way for other parts of the app to read/write data without corrupting it if something goes wrong mid-operation. If this file breaks, the app can't save or load any real data.

`models.py`

CORE

Defines the structure ("shape") of every table in the database — candidates, employees, colleges, form responses, accepted/rejected candidates, email invites, scoring rules, and more. It also enforces two important safety rules: records are never permanently deleted (just marked "deleted" so they can be recovered), and highly sensitive employee data (PAN, Aadhaar, bank account, salary) is stored encrypted rather than as plain text.

`schemas.py`

SUPPORTING

Defines the standard "shape" of a single parsed resume — the specific pieces of information (name, phone, email, work history, education, skills, etc.) extracted from every CV. It's a template that keeps every parsed resume consistently organized so it can be exported, displayed, or saved reliably.

`crypto.py`

CORE

Handles encrypting and decrypting the five most sensitive pieces of employee data — PAN, Aadhaar, UAN, bank account number, and salary/CTC — so they're never stored as plain readable text, even if the database were ever leaked. It relies on a secret passphrase kept outside the database, and also provides a "masking" function so the screen can show "....1234" instead of the full number. Losing the secret passphrase would make this data permanently unreadable, so it must be safely backed up.

`app_paths.py`

CORE

Decides where the app stores its files and data on the computer, whether running as a developer script or as a packaged "double-click" desktop app. For the packaged version, it creates a friendly `HopchargeHR` folder in the user's home directory so a non-technical user always knows where their data lives.

`launcher.py`

CORE

What actually runs when a user double-clicks the HopCharge HR Dashboard icon. It shows a friendly welcome message with the location of data files, starts the dashboard's internal web server quietly in the background, waits until it's ready, and then automatically opens the dashboard in the browser. It also catches crashes and shows a helpful error message (most commonly a missing secrets file) instead of the app silently vanishing.

`launcher_win.py`

MINOR

A small Windows-specific wrapper around `launcher.py`. Since the Windows shortcut hides the console window, a crash would otherwise fail silently; this file catches that and pops up a normal Windows error dialog explaining what went wrong.

`hopcharge.spec`

SUPPORTING

A build recipe used by a packaging tool (PyInstaller) to bundle the entire dashboard — code, web interface, and language model — into a single double-click desktop application for Mac or Windows, so staff don't need to install Python or any technical tools themselves.

2 Resume Reading & Processing

This is the pipeline that turns an uploaded PDF resume into clean, organized candidate data — the very first step every applicant goes through.

`extractor.py`

CORE

Pulls raw text out of an uploaded resume PDF, trying three methods in order of speed and reliability: a fast text-reading tool first, a backup tool if that didn't get enough text, and finally OCR (reading the page like an image) if the PDF is a scan with no selectable text. This ensures the system can get usable text out of almost any resume quality, before handing it to `parser.py`.

`parser.py`

CORE

The "brain" that reads a resume's plain text and figures out the candidate's name, email, phone, location, work history, education, skills, languages, and certifications — turning a messy document into neat, organized data. It uses clever fallback tricks (e.g. cross-checking a guessed name against the email address) to stay accurate even with poorly formatted or scanned resumes, and tags each extracted field with a confidence score. Every resume uploaded to the system passes through this file.

`exporter.py`

SUPPORTING

Saves parsed candidate information into files people can actually open and use: a JSON file (for other software) and nicely formatted Excel/CSV spreadsheets that HR staff can browse, filter, and sort. It turns "data the computer understands" into "reports a person can read."

`forms_retriever.py`

CORE

Connects to Google Forms (the candidate assessment/screening questionnaire) and automatically downloads everyone's answers, matching each response to the right candidate by email. It handles the secure Google connection setup and reserves space in the stored data for an AI to later score each candidate's answers — it's the bridge between "candidate fills out a form" and "AI evaluates the candidate."

`form_tracking.py`

SUPPORTING

Measures how long each candidate takes to fill out the assessment form (a useful evaluation signal) by giving them a unique tracking link that logs a start time before redirecting to the real form, then logging the end time on submission. It also produces summary statistics (average time, fastest/slowest, open-vs-submit rates) for the reporting dashboard.

3 Candidate Pipeline & Records

Once a candidate is in the system, these files track them through every hiring stage — and manage the record that's created once they're actually hired.

`accepted_store.py`

CORE

The "Accepted candidates" filing system. When HR clicks Accept on someone, their full profile moves through four stages in order — HR Round, Round 1, Round 2, and finally Onboarded (hired). It can also auto-move any candidate scoring above a threshold (currently 65) without a manual click, and keeps a full history log of every stage change so nothing is lost if a decision needs reversing.

`rejected_store.py`

CORE

The mirror-image "Rejected candidates" filing system — when HR clicks Reject, the profile and rejection round are saved here, powering the Rejected tab. It also auto-rejects anyone scoring below a threshold (currently 30) during initial screening. Rejections can be undone, and the system never duplicates or overrides a candidate already manually accepted or rejected.

`notes_store.py`

SUPPORTING

A simple running notebook attached to each candidate — HR or interviewers can jot timestamped remarks at any stage. Notes build into a timeline rather than overwriting each other, and are cleared automatically only when a candidate is fully removed from the pipeline.

`employee_db.py`

CORE

The master database for people who have already been hired — stores ID, department, salary, bank info, joining date, and automatically calculates age and length of service. It backs up every change to a Google Sheet automatically, and can import employee lists from a Google Form or a shared spreadsheet. Sensitive fields (PAN, Aadhaar, bank account, salary) are flagged and masked on-screen for privacy.

`status_portal.py`

CORE

Powers the public "Check Application Status" page candidates see via a link in their recruitment email — a simple 4-step progress tracker (Applied → Under review → Interviews → Decision) that never reveals internal scores or rejection reasons. Built with privacy as the priority: unique unguessable links, no way to publicly list candidates, generic "not found" messages for unknown links, and automatic rate-limiting. This is the only part of the system ordinary candidates ever directly touch, so mistakes here carry real privacy risk.

4 Candidate Communication

These files handle every email and message a candidate receives, from the first "thanks for applying" note to the final onboarding welcome.

`emailer.py`

CORE

Sends emails to candidates on Hopcharge's behalf via Gmail: the "please fill out this assessment form" email after applying, and the "Welcome to the Hopcharge Family" onboarding email once hired. It builds branded HTML emails with a plain-text fallback, and pulls the actual wording from `admin_settings.py` so HR can edit messages without touching code.

`calendly_invite.py`

CORE

Invites candidates who passed initial HR screening to book a Round 1 interview via Calendly. It automatically finds everyone who passed, sends a branded "Book your interview" email linking to the correct Calendly page for their specific role, and logs every invite. If a role has no Calendly link configured, it skips that candidate rather than risk sending the wrong booking page.

5 Admin, Settings & Reporting

These files let HR staff customize how the app behaves without touching code, and give leadership a bird's-eye view of recruiting activity.

`admin_settings.py`

CORE

The "settings control panel" — lets HR customize the wording of recruitment/onboarding emails and the score thresholds that decide automatic rejection or advancement (currently below 30 = auto-reject, above 65 = auto-advance). It also stores per-role scoring adjustments (e.g. stricter or more lenient AI grading for a specific job). Changes are saved to a file, with safe built-in defaults if that file is ever missing or corrupted.

`analytics_hr.py`

SUPPORTING

Powers the Analytics dashboard — resumes parsed, emails sent, responses received, accepted/rejected counts, interviews booked — across different time windows and broken down by role and stage. A purely read-only reporting tool that lets management see pipeline health at a glance.

6 Data Sync, Backup & Migration

The app stores data in two places — a Postgres (Neon) database and local JSON backup files. These files make sure the two never fall out of agreement, and handle one-off setup/maintenance tasks like moving to the database in the first place.

`dual_writer.py`

CORE

The "traffic control" file every part of the app goes through to save or load data, keeping the Postgres database and local JSON files in agreement. Postgres is the trusted master copy — written first, with JSON updated only if that succeeds; if the database is briefly unreachable, the app keeps working via JSON alone. Also keeps a short-term memory cache so the dashboard loads instantly. This is why the app doesn't lose data or grind to a halt if the database has a hiccup.

`sync_job.py`

CORE

An automatic reconciliation process that runs every 5 minutes (and on startup) to make sure JSON backup files match the Postgres database. Postgres always wins if the two disagree — but if someone manually edited the JSON file, a safety backup is made first so that edit isn't silently lost.

`drive_sync.py`

SUPPORTING

Lets HR pull in new resumes automatically from a shared Google Drive folder instead of uploading each PDF by hand. It checks for new PDFs, runs them through the standard parsing process, skips duplicates by email, and updates the master candidate list (JSON, CSV, and Excel together).

`migrate_json_to_postgres.py`

CORE · ONE-TIME SETUP TOOL

A one-time script that loaded all pre-existing JSON data (candidates, employees, colleges, form responses, etc.) into the new Postgres database when the app switched from file-based storage to a real database. Safe to re-run without creating duplicates; encrypts sensitive employee data as it goes in.

`verify_migration.py`

SUPPORTING · ONE-TIME SETUP TOOL

A checking tool run right after migration to confirm nothing was lost, duplicated, or corrupted — comparing record counts between old JSON files and the new database, and double-checking that encrypted employee data can still be correctly unlocked. A safety net giving a clear pass/fail before trusting the new database.

`wipe_all_data.py`

SUPPORTING · ADMIN TOOL

A full "factory reset" tool that erases all HR data from both database and JSON files — for testing or starting fresh, not everyday use. Automatically backs up all current files to a timestamped folder first, and requires typing "CONFIRM" as a safeguard against accidental data loss.

schema.sql

CORE

The blueprint of the company's HR database — defines all tables for form applications, parsed resumes, candidate hiring stages, the employee master record, college-outreach contacts, email/link tracking, scoring rules, interview logs, app settings, and the sync ledger. Nothing is ever truly deleted (only marked "deleted"), and sensitive employee details are stored encrypted.

7 AI Candidate Scoring Module (scoring/ folder)

This self-contained folder grades every candidate's written form answers — either using a connected AI service (Claude, OpenAI, Gemini, or Groq) or, if none is configured, a detailed built-in rulebook — and turns the results into a final score and recommendation.

`scoring/engine.py`

CORE

The heart of the scoring system — takes a candidate's form answers and produces a final score and recommendation (e.g. "Strong Shortlist," "Hold," "Reject"). Combines a rule-based score (experience, ratings, yes/no answers) with an AI-judged score (AI grades written answers against a rubric) into one 0–100 total. Lets an HR admin adjust the AI's strictness, and ensures a temporary AI outage never overwrites a candidate's previously good score with a false zero.

`scoring/rubrics.py`

CORE

Stores the actual grading criteria for each role (currently Customer Support Executive and Operations Specialist) based on Hopcharge's real application form: objective checkbox points (40 pts total) plus AI guidance for grading three open-ended questions (60 pts), for a maximum of 100. This is literally "what counts as a good answer" — editing it changes how candidates are judged.

`scoring/rules_engine.py`

CORE

A large, self-contained "offline AI substitute" that scores written answers without any paid AI service, by matching against a huge hand-built dictionary of phrases (formal, casual, Hinglish, texting-style) representing qualities like empathy, ownership, and problem-solving. Used automatically whenever no AI provider is configured, so the company can still screen candidates for free with fair, consistent results. The largest and most detailed file in this folder.

`scoring/providers.py`

CORE

An "adapter" that lets the system talk to different AI companies (Anthropic/Claude, OpenAI, Gemini, Groq) through one consistent set of instructions. Includes a "test connection" feature giving clear, plain-language reasons when something's wrong (invalid key, no credits, wrong model, network blocked) — this is what powers the "Test Connection" button in AI Settings.

`scoring/config_store.py`

CORE

Manages settings for which AI service is used to help score candidates, including safely storing the secret API key in encrypted form. The key is never sent back out to the website — only an internal function can retrieve it to make AI calls. This is the security layer protecting the company's paid AI account credentials.

`scoring/router.py`

CORE

Defines all the web addresses the dashboard calls to use scoring — saving AI settings, testing a connection, viewing/editing rubrics, scoring one or all pending candidates, and pulling analytics/audit history. The "front desk" that routes every scoring-related button-click to the right logic, and automatically moves qualifying candidates into the HR Round stage right after scoring.

scoring/linguistics.py

SUPPORTING

A language-understanding toolkit used mainly in offline (no-AI) mode — detects whether an answer expresses ideas like ownership, apology, or urgency regardless of phrasing (casual English, Hinglish, texting abbreviations). Favors substance over polish, so genuine but badly-worded answers still score reasonably.

scoring/analytics.py

SUPPORTING

Powers the numbers and charts on the scoring dashboard — average score, High Priority vs Reject counts, a score-range breakdown, and average performance per role. Turns raw candidate scores into an at-a-glance summary for hiring managers.

scoring/audit.py

SUPPORTING

Records a permanent log entry (who, when, what role, what score) every time a candidate is scored, without storing actual answers or secret keys — useful for compliance and traceability. Also caps how many scoring requests can run per minute, protecting against a runaway loop generating a huge AI bill.

scoring/time_tracking.py

SUPPORTING

Designed to measure how long a candidate takes filling out the form — overall time, time per question, typing speed, and edit count — to flag patterns like "rushed" versus "careful" responses. Includes ready-to-use tracking code meant for the candidate-facing form, though it is not yet confirmed to be actively wired up to a live form.

scoring/timing_analytics.py

MINOR

Turns the timing data from `time_tracking.py` into summary statistics — average/median completion time, rushed vs. thoughtful counts, slowest form sections, fastest/slowest candidates. Useful for spotting confusing form questions, but depends on timing data that isn't confirmed to be actively captured yet.

scoring/__init__.py

MINOR

A short "table of contents" file that starts the scoring feature and plugs it into the main app. Contains no scoring logic itself.

8 College Outreach Module (colleges/ folder)

This self-contained folder helps the HR team run campus recruitment — finding the right college contacts, prioritizing which colleges to approach first, and tracking outreach progress.

`colleges/directory.py`

CORE

A curated address book of India's top engineering colleges (IITs, NITs, IIITs, BITS, and more) mapping each to its real website and, where known, its actual student-placement page — built because guessing a college's website from its name often fails. It also recognizes nicknames, abbreviations, and typos so staff can type a college name loosely and still get matched correctly.

`colleges/discovery.py`

CORE

The "auto-search" engine that, given a college name, tries to find placement-office contacts by visiting the college's website, checking common pages, and opening any placement brochures. Deliberately avoids scraping LinkedIn and instead hands staff a ready-made search link. Fast (about 9 seconds per college), works on many colleges at once, and remembers results for a few hours — the main time-saver for the outreach team.

`colleges/extract.py`

CORE

The "reading" engine that scans raw web page text and pulls out names, job titles, emails, and Indian phone numbers. Smart about matching a name to the right email even when they're far apart on a page, and avoids merging different staff members who share one generic inbox. Turns messy website text into a clean, reviewable contact list.

`colleges/prioritizer.py`

CORE

The "which college first" calculator — ranks colleges by combining five weighted factors (prestige/tier, student count, internship opportunities, placement office quality, past relationship history) into a 0–100 score labeled High/Medium/Low priority. A transparent, rule-based formula (not a "black box" AI) so staff can see exactly why one college outranks another.

`colleges/services.py`

CORE

The core business logic connecting raw data to the dashboard: tracks a college's progress through the outreach pipeline (logging dated notes on status changes), builds the summary statistics shown on dashboard cards and charts, and handles bulk import/export via CSV or Excel — including cleaning up messy column names from uploads.

`colleges/store.py`

CORE

The "filing cabinet" that saves and retrieves all college records (currently a JSON file on disk). Prevents duplicate colleges by recognizing that "IIT Delhi," "I.I.T. Delhi," and "iit delhi" are the same institution, and safely handles simultaneous edits. Every create/update/delete/import action from other files passes through here.

`colleges/schemas.py`

CORE

Defines the "shape" of a college record — every field tracked (name, contact person, email, phone, location, outreach stage, priority score, notes) and the allowed outreach stages (e.g. "Not Contacted" → "Email Sent" → "Interested" → "Active Partner"). The official template keeping every college's data consistent.

`colleges/router.py`

CORE

Defines every web address that lets the dashboard interact with college data — creating, listing, editing, deleting records, advancing outreach stages, recalculating priority scores, importing/exporting spreadsheets, and running auto-discovery. The traffic-control layer connecting the interface to every other file in this folder.

`colleges/status.html`

SUPPORTING

A public webpage design letting a job applicant check their application status via a personal emailed link — name, role applied for, a visual progress timeline, and a "what happens next" message, styled with Hopcharge branding. Despite living in this folder, it serves the broader recruitment process rather than college outreach specifically.

`colleges/enrichment.py`

MINOR

A placeholder for a possible future paid service (like Hunter.io) that could look up a person's work email from just their name and college. Currently inactive by default and does nothing unless a subscription and API key are added later.

`colleges/sample_colleges.csv`

MINOR

A small sample spreadsheet with 5 example colleges (IIT Delhi, NIT Trichy, IIIT Hyderabad, VIT Vellore, BITS Pilani) showing the expected columns — college name, type, city, contact details, website, intake size, engagement level. A template/demo for testing the import feature.

`colleges/__init__.py`

MINOR

A short "table of contents" file that plugs the whole College Outreach module into the main HR system in one line, and notes that a planned future feature (automatically crawling college websites) hasn't been built yet.

`colleges/requirements.txt`

MINOR

A shopping list of external software components (PDF readers, Excel writers, web-request tools) needed for the college module to run. Purely technical setup, not something a business user would open.

9 Setup, Installers & Branding

These files control how the app gets installed on a new computer, what software it depends on, and how it looks.

`requirements.txt`

SUPPORTING

Lists all the third-party software building blocks the program needs: the web server that powers the dashboard, PDF-reading tools (including OCR for scanned resumes), a language-processing library for understanding resume text, spreadsheet/Excel export tools, the Postgres database connector, encryption tools, and Google Forms/Drive/Sheets connectors. A shopping list of pre-built pieces the app is assembled from.

`Install-HopCharge.bat`

CORE

The one-click Windows installer. When double-clicked, it checks for and installs Python if needed, sets up a private self-contained copy of everything the app needs, downloads the language model used to read resumes, creates the app's data folders, and adds a "HopCharge HR Dashboard" shortcut to the Desktop and Start Menu.

`README-Windows-Setup.md`

SUPPORTING

The instruction sheet for installing the dashboard on Windows — how to run the installer, click through a security warning, find the created shortcut, and where to place the two private config files (database settings and Google credentials). Notes the app still works without those files, just saving data locally instead of to the cloud database.

launcher / folder

`launcher / Install-HopCharge-App.command`

SUPPORTING

The Mac equivalent of the Windows installer — when double-clicked, builds a proper "HopCharge HR.app" application on the Desktop so the dashboard can be launched like any normal Mac program, no technical knowledge required. The folder also holds the Mac app icon image.

`appicon.ico`

MINOR

The small icon image used for the app's shortcut and window on Windows.

`static/index.html`

CORE

The actual HR dashboard screen staff see and use in their browser — includes candidate management (accept/reject/AI scoring), college outreach and application-form tracking, analytics/reports, employee management, and settings screens. This is the entire visual interface of the tool.

`static/Assets/` & `static/hopcharge-white.png`

MINOR

The HopCharge company logo images (navy and white versions) used to brand the dashboard interface and any generated emails or documents.

hopcharge.spec

SUPPORTING

Listed also under Section 1 — the PyInstaller build recipe that bundles the app into a single installable program for Mac or Windows.

10 Data, Credentials & Log Folders

These are not code files but the folders where the app actually stores information, secrets, and activity records while running.

`output/`

CORE

Where the app stores all of its working data locally, organized into groups: **candidate data** (accepted/rejected candidates, resumes, CSV/Excel exports), **settings** (AI settings, admin settings, Google Drive/Sheets sync settings, tracking configuration), and **logs/audit trails** (AI decision records, form-tracking activity). The subfolder `_backup_before_wipe_20260626_113146/` is an automatic safety snapshot the system made before a reset action, so nothing would be permanently lost if that reset needed undoing.

`credentials/`

CORE

The designated storage spot for private login/connection files the app needs (such as Google service-account files). Currently empty in this copy, but its contents — whenever present — should always be treated as strictly confidential.

`input_resumes/`

CORE

The "drop box" folder where PDF resumes are placed (or automatically saved) for the app to read and process. Currently empty, meaning no resumes are waiting to be processed right now.

`neon.env`

CORE · SECRET

Stores the secret connection details that let the app connect to its online (Neon) database. Contains sensitive data and must never be shared, emailed, or uploaded anywhere public.

`Gdrive service acc.json`

CORE · SECRET

A secret digital "ID card" that lets the app log into Google Drive automatically without a person typing a password. Contains sensitive credentials and should be kept private.

`Gforms service acc.json`

CORE · SECRET

A secret digital "ID card" that lets the app log into Google Forms automatically to pull in job-application responses. Contains sensitive credentials and should be kept private.

`launcher_server.log`

MINOR

A log file recording what happened the last time the app's launcher was started — useful for troubleshooting if the app fails to open.

`.claude/settings.local.json`

MINOR

Configuration for the Claude Code AI assistant used while building/maintaining this project — controls what actions the assistant may take on this computer. Not part of the HR dashboard app itself.

.dist/

MINOR

An empty folder reserved for finished, packaged versions of the app produced by the build/installer tools. Currently empty.

Housekeeping note: This dev folder (`BAU Automation- Hopcharge`) is the source of truth for the app's code. Three separate packaged copies exist elsewhere for distribution to Windows and Mac users — code changes made here should be copied forward to those, while secrets and installer scripts are kept separate per copy.